

CYBERSECURITY INTERNSHIP REPORT

Redynox — Cyber Security Internship Program

Intern Name: Sumit Jaiswal

Intern ID: RDXINTSUMIW89MD

Role: Cyber Security Intern

Duration: 6 August 2026 – 5 September 2026 (1 Month)

Mode: Remote

Reporting Manager: Ankit Mathur, Internship Supervisor/Lead

Academic Background: B.Tech CSE (Cybersecurity Specialization), Final Year

Report Date: 5 September 2026

1. Introduction

During my internship, I worked on practical tasks related to cybersecurity fundamentals and web application security. The purpose was to gain hands-on experience in identifying threats, implementing basic security measures, and analysing vulnerabilities in both networks and web applications. These tasks helped me strengthen my technical foundation and apply theoretical knowledge to real-world scenarios.

2. Task Overview

2.1 Task 1: Introduction to Network Security Basics

- **Objective:** To understand the basics of network security by learning about different types of network threats, and to implement basic security measures that help protect a small network.
- **Tools & Methods Used:** Windows Defender Firewall, Wireshark, home Wi-Fi router settings, WPA2/WPA3 encryption.

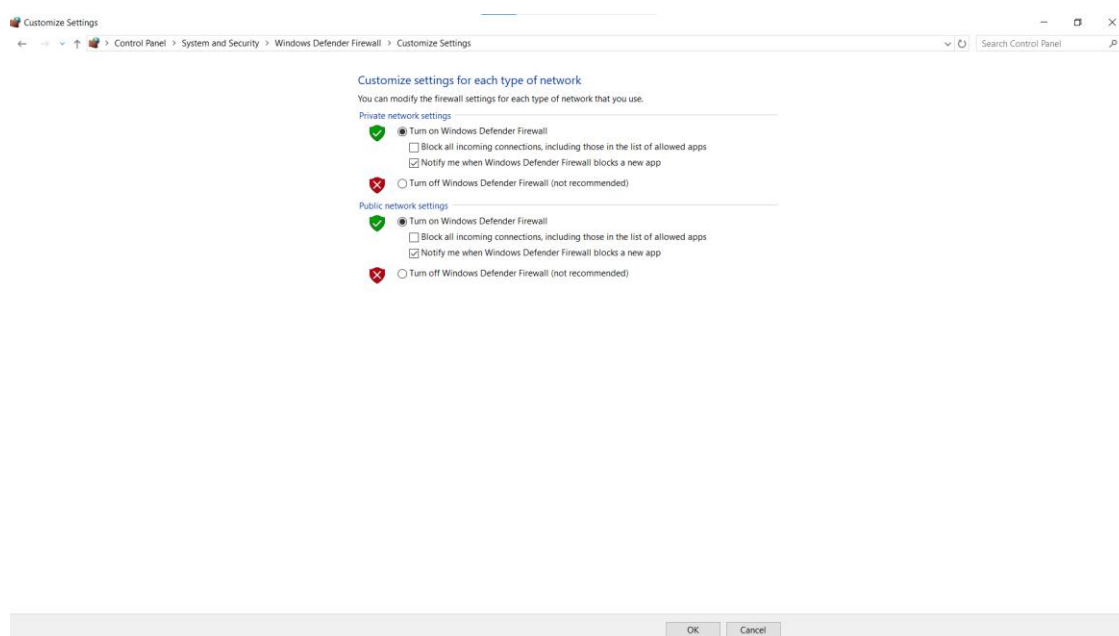
2.1.1 Understanding Network Threats

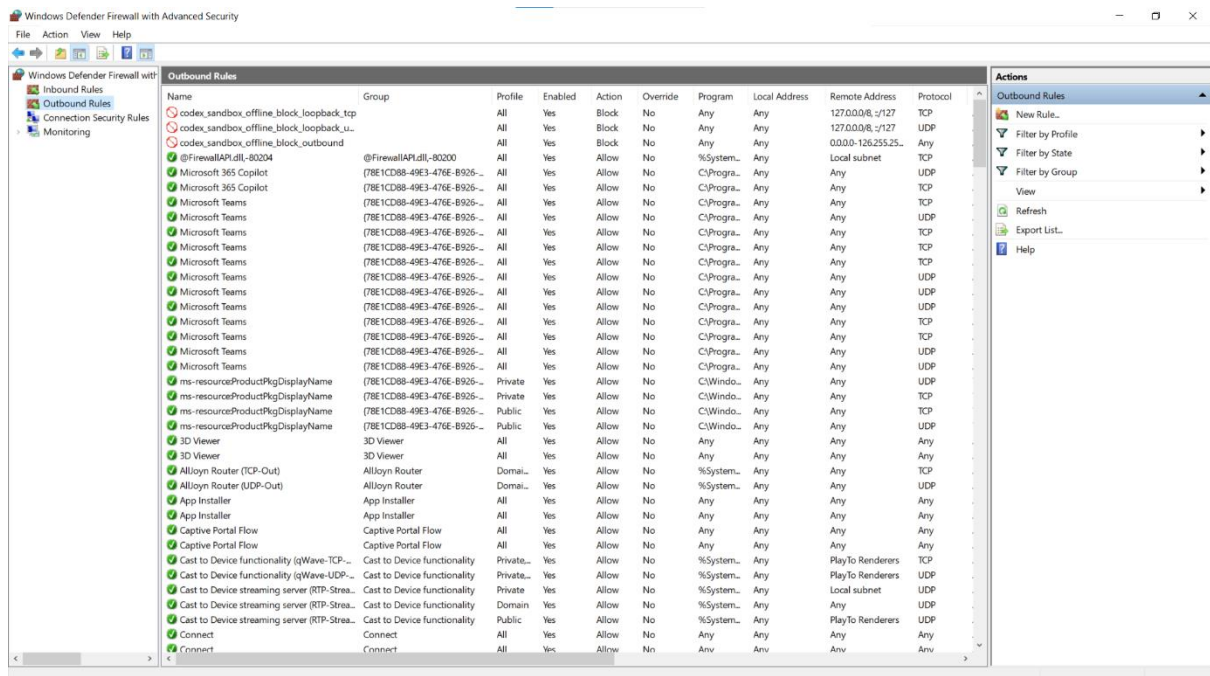
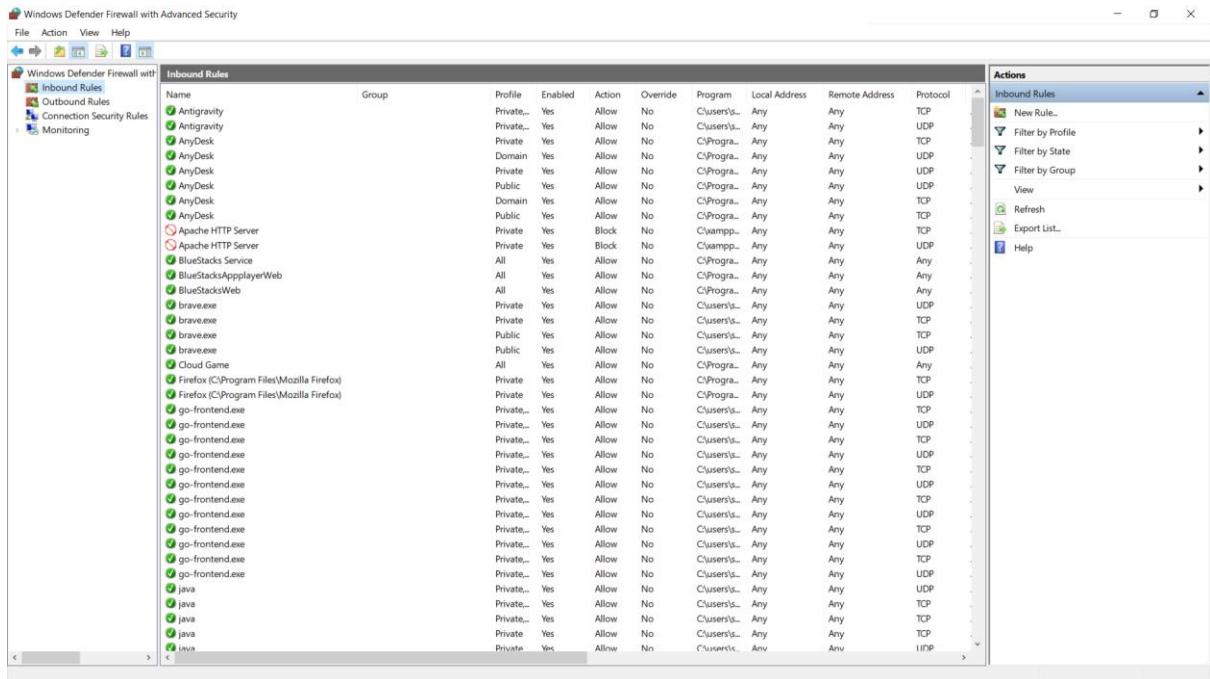
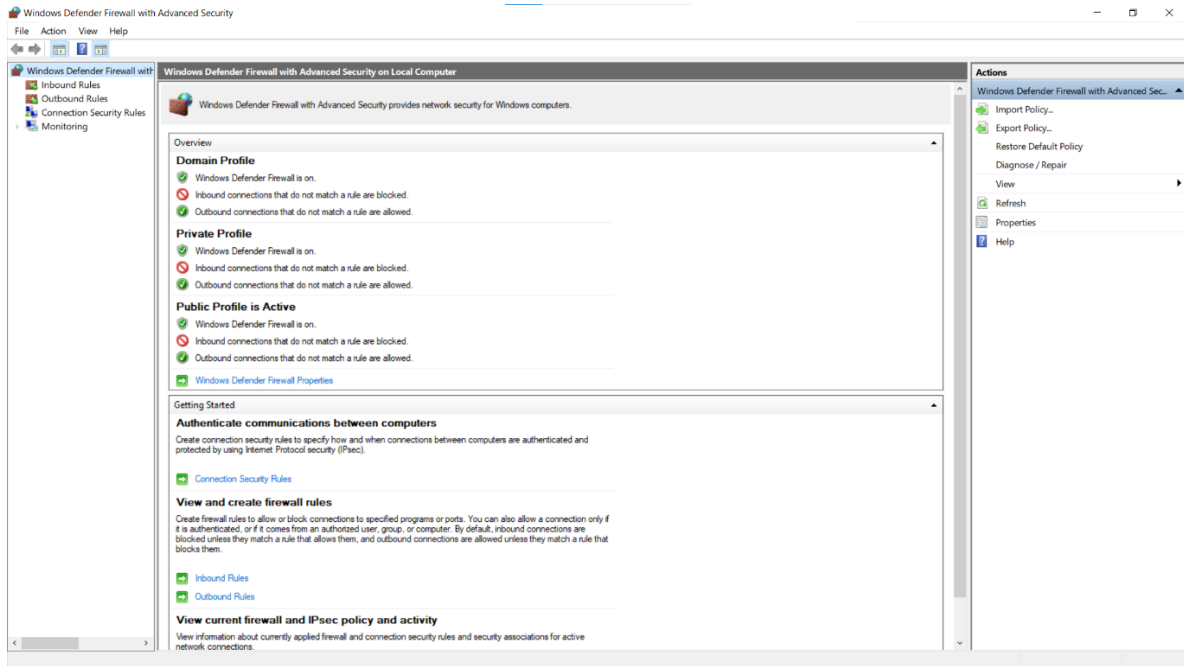
Before making any changes to my network, I first spent time understanding the common threats that a home or small office network usually faces. A virus is a piece of malicious code that attaches itself to a genuine file or program and spreads when that file is shared or opened. A worm is similar, but it does not need a host file — it can copy itself and spread across a network on its own, often slowing things down or opening backdoors for attackers. A trojan disguises itself as a normal, useful program, but once installed, it quietly gives an attacker access to the system in the background. Phishing is different from the other three because it does not rely on code at all — it relies on tricking the user, usually through a fake email or website that looks real, into giving away passwords, OTPs, or card details. Understanding these differences helped me see why network security is not just about installing one tool, but about combining several layers of protection — a firewall, encryption, and safe user habits — together.

2.1.2 Setting Up the Firewall

The first practical step I took was to check and properly configure the firewall on my system. I opened Windows Defender Firewall and made sure it was switched on for both private and public network profiles, since leaving it off — even temporarily — exposes the system to unwanted inbound connections. I went through the existing inbound and outbound rules, removed the ones that looked unnecessary or too permissive, and confirmed that unauthorized programs were being blocked from accessing the network. This step alone showed me how much of network security is just making sure the basic protections that are already built into the operating system are actually switched on and configured correctly, instead of being left on default settings.

Figure 1: Windows Defender Firewall — showing the firewall turned ON for the network profile, with inbound/outbound rules visible





2.1.3 Securing the Router (Passwords & Encryption)

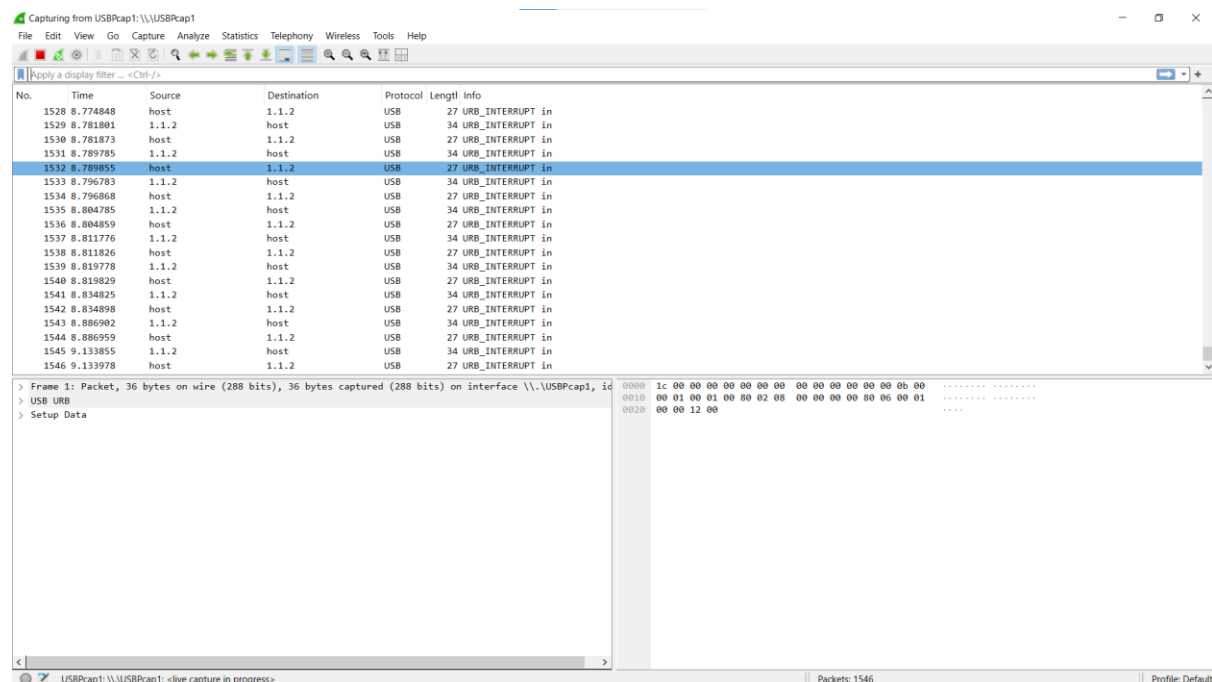
I was not able to access the router's admin panel directly for this task, since I am connected to a college/shared network where router-level settings are managed by the network administrator and are not accessible to individual users. Because of this, I focused on understanding and documenting how router-level password and encryption settings contribute to network security, rather than making live changes myself.

Every Wi-Fi router ships with a default admin username and password, and leaving these unchanged is one of the most common ways attackers gain control of a network, since default credentials for most router brands are publicly known. Changing this to a strong, unique password closes off that entry point. Separately, the wireless encryption standard (WPA2 or the newer WPA3) determines how the data travelling between a device and the router is scrambled, so that anyone else on the same radio range cannot simply read the traffic. WPA2 uses AES encryption and is still considered secure for most home and small office use; WPA3 improves on it further with stronger encryption and better protection even when a weak password is used. Using the older WEP standard, or no encryption at all, would leave the network open to anyone nearby.

On a personal home network, I would apply these two settings by logging into the router's admin panel (usually at an address like 192.168.1.1 or 192.168.0.1), changing the default admin password under Administration settings, and then enabling WPA2 or WPA3 under the Wireless Security settings.

2.1.4 Capturing and Analysing Traffic with Wireshark

To actually see what was happening on the network rather than just assuming it was secure, I installed Wireshark and ran a live packet capture while browsing normally. I applied simple filters to isolate specific protocols, mainly http and dns, so the capture would not be overwhelming to read. This let me see, in plain terms, which websites my system was resolving through DNS, and which requests were still being sent over plain, unencrypted HTTP instead of HTTPS. I did not find anything malicious in my own capture, but the exercise made it clear how easily traffic can be observed if someone has access to the same network, and why using HTTPS-only websites and a trusted, encrypted network matters as much as having a firewall.



Capturing from Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

No.	Time	Source	Destination	Protocol	Length	Info
1	21.42448	2409:40b0:247b:aadf::64:ff9b::d294:551e	2409:40b0:247b:aadf::	HTTP	469	POST /statistics/clienttype=&version=1.0.0&ver=1 HTTP/1.1
2	40.2941	64:ff9b::d294:551e	2409:40b0:247b:aadf::	HTTP	463	HTTP/1.1 200 OK (text/html)

> Frame 367: Packet, 469 bytes on wire (3752 bits), 469 bytes captured (3752 bits) on interface \Device\NPF...
> Ethernet II, Src: AzureWaveTec_d1:16:23 (08:c5:d3:d1:16:23), Dst: 96:ca:42:1f:f5:dd (96:ca:42:1f:f5:dd)
> Internet Protocol Version 6, Src: 2409:40b0:247b:aadf::f020:f4cc:2f69:33f6, Dst: 64:ff9b::d294:551e
> Transmission Control Protocol, Src Port: 54842, Dst Port: 80, Seq: 300, Ack: 1, Len: 395
> [2 Reassembled TCP Segments (694 bytes): #365(299), #367(395)]
> Hypertext Transfer Protocol

0000 96 ca 42 1f f5 dd d0 c5 d3 d1 16 23 86 dd 60 0e ...B.....
0010 71 64 81 9f 06 3f 24 09 40 d0 24 7b aa ff f0 20 ...qd...7\$...@S{...
0020 f4 cc 2f 69 33 f6 00 64 f9 0b 00 00 00 00 00 00 .../13-d
0030 00 00 02 94 55 1e d0 3a 00 50 7f fe 04 08 b1 74 ...-0- P...t
0040 40 b4 50 18 01 04 0b 69 00 00 2d 2d 2d 2d 4d 55 ...@ P...:MU
0050 4c 54 49 2d 50 41 52 54 53 2d 46 4f 52 4d 2d 44 ...LTI-PART 5-FORM-D
0060 41 54 41 2d 42 4f 55 4e 44 41 52 59 0d 0a 43 6f ...ATA-BOUR DAIRY Co
0070 6e 74 65 6e 74 2d 44 69 73 70 6f 73 69 74 69 6f ...ntent-Di spositio
0080 6e 3a 20 66 6f 72 6d 2d 64 61 74 61 3b 20 6e 61 ...n form: data; na
0090 6d 65 3d 22 75 73 65 72 66 69 6c 65 22 3b 20 66 ...me="user file"; f
00a0 69 6c 65 6e 61 6d 65 3d 22 66 69 6c 65 22 0d 0a ...llemame="file".
00b0 43 6f 6e 74 65 6e 74 2d 54 79 70 65 3a 20 61 70 ...Content-Type: ap
00c0 70 6c 69 63 61 74 69 6f 6e 2f 6f 63 74 65 74 2d ...plicitation/octetet
00d0 73 74 72 65 61 6d 0d 0a 0d 0a 24 94 0e c9 81 ab ...stream: ...\$...
00e0 1c a9 3f be a9 63 47 08 f8 8d 0b 4c db e7 d6 45 ...-?..cG...L...E
00f0 d8 18 22 b2 bc 06 ce 42 45 57 73 44 44 7d ae 4a ...-...B Em6 D)3
0100 df c4 1e 80 63 c0 e6 25 8f 42 17 04 78 d2 2f 66 ...-...c..B...a..f
0110 0a 2a d2 5a 7e 8c 64 77 86 5c 64 f5 8f 8b 80 f5 ...* Z...du \d...
0120 6e 83 42 c2 8c e7 5e aa 70 17 6e e5 5e 6d 7c cd ...n B...p n...
0130 6c 80 a6 f4 fd 18 a2 2c d8 53 aa 5c 82 78 ed ...1...S...c...
0140 20 5d 7d 22 de 41 16 32 41 b6 e1 5a e6 20 2b 0e ...})] A 2 A -Z...+
0150 07 72 1f 99 58 e7 e2 00 6b 7a 4a 2b 7e 2f 8e 56 ...-X...k23+/-V
0160 53 86 29 c5 30 60 ad 7e 45 67 10 ae 0a 77 5f 40 ...S...@...g...w@

Packet (469 bytes) Reassembled TCP (694 bytes)

HyperText Transfer Protocol: Protocol Packets: 2869 - Displayed: 2 (0.1%) Profile: Default

Capturing from Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

dns

No.	Time	Source	Destination	Protocol	Length	Info
1	125.9	0.839901	10.147.71.114	DNS	71	Standard query 0x3355 AAAA discord.com
2	126.9	0.839901	10.147.71.114	DNS	71	Standard query 0x3355 AAAA discord.com
3	127.9	0.839901	10.147.71.114	DNS	120	Standard query response 0x3355 AAAA discord.com
4	128.9	0.839901	10.147.71.114	DNS	211	Standard query response 0x3355 AAAA discord.com
5	129.9	0.839901	10.147.71.114	DNS	151	Standard query response 0x3355 AAAA discord.com
6	166.9	0.569935	10.147.71.114	DNS	72	Standard query 0x1b77 A nephobox.com
7	167.9	0.569935	10.147.71.114	DNS	72	Standard query 0x1b77 A nephobox.com
8	169.9	0.569935	10.147.71.114	DNS	88	Standard query response 0x1b77 A nephobox.com
9	170.9	0.569935	10.147.71.114	DNS	100	Standard query response 0x1b77 A nephobox.com
10	223.14	10.1547	10.147.71.114	DNS	87	Standard query 0x3355 AAAA discord.com
11	224.14	10.1547	10.147.71.114	DNS	87	Standard query 0x3355 AAAA discord.com
12	225.14	10.1547	10.147.71.114	DNS	303	Standard query response 0x3355 AAAA discord.com
13	226.14	10.1547	10.147.71.114	DNS	351	Standard query response 0x3355 AAAA discord.com
14	280.17	0.837575	10.147.71.114	DNS	77	Standard query 0x9ba1 HTTPS tzm.protechs.net
15	281.17	0.837575	10.147.71.114	DNS	77	Standard query 0x9ba1 HTTPS tzm.protechs.net
16	282.17	0.837575	10.147.71.114	DNS	77	Standard query 0x9ba1 HTTPS tzm.protechs.net
17	283.17	0.837575	10.147.71.114	DNS	197	Standard query response 0x9ba1 HTTPS tzm.protechs.net
18	285.17	0.837575	10.147.71.114	DNS	132	Standard query response 0x9ba1 HTTPS tzm.protechs.net
19	286.17	0.837575	10.147.71.114	DNS	132	Standard query response 0x9ba1 HTTPS tzm.protechs.net

> Frame 332: Packet, 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface \Device\NPF...
> Ethernet II, Src: 96:ca:42:1f:f5:dd (96:ca:42:1f:f5:dd), Dst: AzureWaveTec_d1:16:23 (08:c5:d3:d1:16:23)
> Internet Protocol Version 4, Src: 10.147.71.95, Dst: 10.147.71.114
> User Datagram Protocol, Src Port: 53, Dst Port: 52829
> Domain Name System (response)

0000 d0 c5 d3 d1 16 23 96 ca 42 1f f5 dd 08 00 45 00 ...-...B.....E
0010 00 51 09 43 40 00 40 11 8d 62 8a 93 47 5f 0a 93 ...Q C@ @ -b -G...
0020 47 72 00 35 ca 5d 00 3d 34 c7 3b 9f 81 80 00 01 ...Gr S] = 4...
0030 00 01 00 00 00 00 00 61 63 63 6f 75 6e 74 73 06 ...-...a ccounts
0040 67 6f 6f 67 6c 65 03 63 6f 6d 00 00 01 00 c0 ...google.c om.....
0050 0c 00 01 00 01 00 00 01 8c 00 04 c0 b2 9e 54 ...-...T

Domain Name System: Protocol Packets: 3678 - Displayed: 170 (4.6%) Profile: Default

Capturing from Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Current filter: dns

No.	Time	Source	Destination	Protocol	Length	Info
1	227.7	2.234885	10.147.71.114	DNS	141	Standard query response 0x2d68 HTTPS signalr-pa.clients6.google.com
2	276.8	2.212930	10.147.71.114	DNS	75	Standard query 0xb7c4 HTTPS ssl.gstatic.com
3	277.8	2.214039	10.147.71.114	DNS	75	Standard query 0xb7c4 HTTPS ssl.gstatic.com
4	278.8	2.215022	10.147.71.114	DNS	75	Standard query 0xb7c4 HTTPS ssl.gstatic.com
5	279.8	2.215022	10.147.71.114	DNS	103	Standard query response 0xb7c4 HTTPS ssl.gstatic.com
6	287.8	2.247748	10.147.71.114	DNS	132	Standard query response 0xb7c4 HTTPS ssl.gstatic.com
7	288.8	2.248888	10.147.71.114	DNS	91	Standard query response 0xb7c4 HTTPS ssl.gstatic.com
8	338.9	2.282796	10.147.71.114	DNS	78	Standard query 0x0eff HTTPS history.google.com
9	339.9	2.282796	10.147.71.114	DNS	78	Standard query 0x0eff HTTPS history.google.com
10	340.9	2.282796	10.147.71.114	DNS	78	Standard query 0x0eff HTTPS history.google.com
11	341.9	2.282796	10.147.71.114	DNS	152	Standard query response 0x0eff HTTPS history.google.com
12	342.9	2.282796	10.147.71.114	DNS	214	Standard query response 0x0eff HTTPS history.google.com
13	343.9	2.282796	10.147.71.114	DNS	198	Standard query response 0x0eff HTTPS history.google.com
14	369.10	2.16679	10.147.71.114	DNS	74	Standard query 0xb044 HTTPS ws.chatgpt.com
15	370.10	2.17623	10.147.71.114	DNS	74	Standard query 0xb044 HTTPS ws.chatgpt.com
16	371.10	2.18476	10.147.71.114	DNS	74	Standard query 0xb044 HTTPS ws.chatgpt.com
17	372.10	2.255091	10.147.71.114	DNS	119	Standard query response 0xb044 HTTPS ws.chatgpt.com
18	373.10	2.257249	10.147.71.114	DNS	102	Standard query response 0xb044 HTTPS ws.chatgpt.com
19	374.10	2.257249	10.147.71.114	DNS	106	Standard query response 0xb044 HTTPS ws.chatgpt.com

> Frame 1: Packet, 79 bytes on wire (632 bits), 79 bytes captured (632 bits) on interface \Device\NPF...
> Ethernet II, Src: AzureWaveTec_d1:16:23 (08:c5:d3:d1:16:23), Dst: 96:ca:42:1f:f5:dd (96:ca:42:1f:f5:dd)
> Internet Protocol Version 4, Src: 10.147.71.114, Dst: 10.147.71.95
> User Datagram Protocol, Src Port: 52369, Dst Port: 53
> Domain Name System (query)

0000 96 ca 42 1f f5 dd d0 c5 d3 d1 16 23 08 00 45 00 ...-...B.....E
0010 00 41 40 2a 00 00 00 11 50 8b 0a 93 47 72 8a 93 ...-...P...G...
0020 47 5f cc 93 00 35 00 2d 4d 00 c0 0d 01 00 00 01 ...G...S...R...
0030 00 00 00 00 00 00 00 63 6f 69 65 6e 74 73 34 06 ...-...c lients
0040 67 6f 6f 67 6c 65 03 63 6f 6d 00 00 01 00 01 ...google.c om...A...

Wi-Fi -> live capture in progress Packets: 398 - Displayed: 42 (10.6%) Profile: Default

2.1.5 Documentation & Reflection

Putting this together, the network threats I researched (viruses, worms, trojans, phishing) each attack a system in a different way, so no single tool covers all of them — a firewall stops unauthorized connections, WPA2/WPA3 encryption stops eavesdropping on the Wi-Fi signal itself, and traffic monitoring with Wireshark helps confirm that both of those are actually working as expected. For a larger, more complex network, I would additionally look at network segmentation (keeping guest devices on a separate network from work devices), an Intrusion Detection/Prevention System (IDS/IPS), and a VPN for any remote access, since a single firewall is not enough once there are many more devices and entry points to protect.

If I had to explain this to someone with no technical background, I would keep it to three habits: always use a strong, unique password (never the default one that comes with a device), always check that a Wi-Fi network is encrypted before connecting, and never click a link or attachment from an unknown or unexpected sender. Most real-world attacks succeed not because of some advanced exploit, but because one of these three basic habits was skipped.

- **Challenges Faced:** Reading raw Wireshark output was confusing at first since a single page load generates dozens of packets; this got easier once I applied protocol filters (http, dns) and focused on one conversation at a time instead of the full capture.
- **Results & Outcomes:** By the end of this task, my system had a properly enabled and configured firewall (verified through both the basic Windows Security status and the detailed Inbound/Outbound Rules list in Advanced Security), along with a documented Wireshark capture showing filtered HTTP and DNS traffic. This gave me a clearer, hands-on understanding of how firewall rules and traffic monitoring work together to protect a network, even without direct access to router-level settings.